

H5. 자기상관

어제와 오늘의 답음을 숫자 하나로 쟀다 — 그리고 "내일도 오늘
같겠지"가 왜 의외로 강한지

지난 시간 복습 — H4

- 관측을 추세 + 계절 + 잔차로 쪼갬다. 계절을 빼고 남긴 편차와 잔차는 쓰레기가 아니라 날씨 사건의 기록이었다.
- H4 마지막 문제: 잔차가 하루마다 제멋대로 널뛰지 않고, 한파처럼 며칠씩 뭉쳐 다녔다 — "어제가 -5면 오늘도 음수일 것 같다".
- 오늘은 그 "어제와 오늘의 닮음"을 숫자 하나로 잴다 — 자기상관(autocorrelation)이다.

오늘의 목표

- shift로 1,096일 전체의 (어제, 오늘) 짝을 만들 수 있다.
- **lag plot**을 그리고 읽을 수 있다.
- 자기상관을 정의하고, 원본과 편차에서 각각 잴 수 있다 — 계절이 상관을 부풀리는 착시를 설명할 수 있다.
- **ACF**를 그려 "기억이 며칠 가는지"와 데이터의 주기를 읽을 수 있다.
- "내일도 오늘 같겠지"(naive 예측)가 왜 강한지 자기상관의 언어로 말할 수 있다.

손으로 워밍업 — 한파 주간의 어제와 오늘

H4에서 만든 편차의 2026년 1월 한파 주간 값. (단위: °C)

날짜	1/18	1/19	1/20	1/21	1/22	1/23	1/24	1/25
편차	+2.2	-1.5	-7.2	-7.8	-7.4	-4.8	-4.1	-5.0

- 이 8일에서 (어제의 편차, 오늘의 편차) 짝 7개가 나온다 — 첫 두 짝은 (+2.2, -1.5), (-1.5, -7.2).
- 나머지 짝을 채우고 모눈에 찍는 것은 활동지에서 — 점들이 어느 구역에 모이는지 직접 본다.

상관계수 복습 — 언제나 두 목록 사이

- 고교 통계의 상관계수 r : 두 목록이 같이 움직이는 정도 — +1이면 완벽하게 같이, 0이면 무관, -1이면 정반대.
- 상관은 언제나 두 목록 사이에서 구한다 — 키 목록과 몸무게 목록처럼, 같은 길이로 짝지어진 두 줄.
- 오늘의 아이디어: 두 번째 목록을 남이 아니라 자기 자신의 과거로 삼으면 어떻게 될까? — "어제의 기온 목록" vs "오늘의 기온 목록".

shift 읽는 법 — 오늘 자리에 어제 값

```
t.shift(1)
```

= t의 모든 값을 하루씩 뒤로 밀어라(shift)

— 그러면 각 날짜 자리에 "어제의 값"이 와 있다

날짜	t	t.shift(1)
8/1	28.1	NaN
8/2	27.3	28.1
8/3	26.7	27.3

- 첫날 자리는 어제가 없으므로 **NaN** — H3의 NaN과 같은 정직함.
- 헛갈림 주의: shift(1)은 값을 **미래 쪽으로** 민다. 그래서 오늘 자리에 어제 값이 온다. (shift(-1)이 반대 방향.)

lag plot — 1,095쌍의 어제와 오늘

```
pair = pd.DataFrame({"어제": t.shift(1), "오늘": t})  
pair.plot.scatter(x="어제", y="오늘", s=2)
```

- 워밍업 모눈의 7점을 pandas로 1,095쌍 전부 찍은 것이다.
- 이렇게 "k일 전 vs 오늘"을 흩뿌린 그림을 **lag plot**이라 부른다
— 시차(lag)는 며칠 밀었는지를 뜻한다.
- 점들이 어떤 모양으로 모이는지, 그 모양이 무슨 뜻인지는
활동지에서 직접 읽는다.

자기상관 — 상대가 자기 자신이다

```
pair["어제"].corr(pair["오늘"])
```

= "어제" 목록과 "오늘" 목록 사이의 상관계수 r 을 구하라
(corr는 correlation의 줄임말)

- 키 vs 몸무게 같은 "남"과의 상관이라 아니라, 상대가 자기 자신을 하루 민 복사본(`t.shift(1)`)이다 — 그래서 이 값을 자기상관이라 부른다.
- 첫날의 NaN 걱정은 필요 없다 — corr는 짝이 안 맞는 줄을 알아서 빼고, 1,096일에서 1,095쌍으로 계산한다.

계절의 착시 — 닳음의 절반은 거저 얻은 것

- 여름날의 어제는 어차피 여름날이다 — 8월의 아무 이틀을 골라도, 이웃한 날이 아니어도 서로 닳는다.
- (어제, 오늘) 짝은 언제나 같은 계절 안의 이틀이므로, 계절이 만든 닳음이 모든 짝에 얹혀 상관을 부풀린다.
- 진짜 질문: 계절을 빼고도 어제가 오늘을 말해 주는가?
- 계절을 뺀 시계열은 이미 있다 — H4의 편차 **a**(그날 기온에서 그 달의 보통을 뺀 것). 같은 lag plot과 상관을 a로 반복해서, 얼마나 줄고 무엇이 남는지 활동지에서 확인한다.

autocorr 읽는 법 — 시차 k의 자기상관

그제와는? 사흘 전과는? — shift(2), shift(3), ...을 일일이 쓰는 대신 줄임말이 있다:

```
a.autocorr(k)
```

= `a.corr(a.shift(k))`의 줄임말 — k일 전과의 자기상관
(k에 1, 2, 3, ...을 넣어 가며 "기억의 세기"를 잴 수 있다)

```
lags = range(1, 31)
acf_a = pd.Series([a.autocorr(k) for k in lags], index=lags)
acf_a.plot(label="편차의 자기상관", legend=True)
```

- 대괄호 안은 "k를 1부터 30까지 바꿔 가며 모아 담아라"는 뜻 — 문법 연습이 아니라 도구 사용이다.

ACF — 시차별 자기상관의 목록

- 방금 만든 것이 **ACF**(autocorrelation function)다. 새 계산이 아니다 — 시차 1의 자기상관 하나를 시차 2, 3, ...으로 반복해 답을 시차 순서로 늘어놓은 것.
- 시차를 넣으면 자기상관이 나오는 대응이라서 함수(function)라 부르고, 가로축 시차·세로축 자기상관의 곡선으로 그린다.
- 참고: `autocorr(0)` 은 자기 자신과의 상관이므로 정확히 1 — ACF는 항상 1에서 출발해 내려온다. 우리 그래프는 시차 1부터 그렸을 뿐이다.

ACF 읽는 법 — 기억은 며칠 가나

- 곡선의 각 점은 "**k일 전을 알면 오늘을 얼마나 말할 수 있나**"다 — 워밍업에서 손으로 세운 질문을 시차별로 반복한 것.
- 값이 크면 그 시차의 과거가 오늘을 많이 말해 주고, **0** 근처까지 내려간 시차의 과거는 아무것도 말해 주지 못한다 — 그만큼 먼 과거를 시계열이 잊은 것이다.
- 그래서 곡선이 0에 닿는 시차가 곧 **기억의 길이다**. 서울 날씨의 기억이 며칠인지는 활동지에서 직접 읽는다.

ACF는 주기의 탐지기이기도 하다

- 주기가 있는 시계열은 주기만큼 시간이 지나면 같은 국면이 돌아온다 — 그래서 그 시차에서 값이 되살아나 ACF에 물결이 나타난다.
- 봉우리가 서는 시차를 읽으면 그 데이터의 달력이 보인다 — ACF는 값의 측정기이면서 동시에 주기의 탐지기다.
- 편차가 아니라 원본 t 의 ACF를 시차 1년까지 그리면 무엇이 보일까 — 바닥과 꼭대기가 어느 시차에 서는지 활동지에서 확인한다.

naive 예측 — 가장 게으른 전략

내일 기온을 숫자 하나로 말해야 한다. 두 전략:

- (a) 3년 전체 평균 **12.7°C**를 말한다.
- (b) 오늘 기온을 그대로 말한다 — 이 게으른 전략을 **naive 예측**(naive forecast)이라 부른다.

채점은 "평균적으로 몇 도 틀리는가":

```
(t - t.mean()).abs().mean()      # 매일 평균을 말했다면  
(t - t.shift(1)).abs().mean()    # 매일 어제 기온을 말했다면
```

- 누가 몇 배나 이기는지, 그 이유를 오늘 배운 말로 어떻게 설명하는지 — 활동지에서.

오늘 배운 세 문장

1. 자기상관은 자기 자신을 k 일 민 복사본과의 상관계수다 — 어제와 오늘의 닮음을 재는 숫자.
2. 원본의 자기상관에는 계절의 공로가 섞여 있다 — 계절을 빼고 남는 것이 날씨의 진짜 뭉침이다.
3. 자기상관이 큰 데이터에서는 가장 가까운 과거가 가장 좋은 예측 재료다.

이제 활동지로

1. 한파 주간 8일로 (어제, 오늘) 짝 7개를 만들어 모눈에 찍는다.
2. shift로 1,095쌍의 lag plot을 그리고, 상관계수를 구한다.
3. 같은 일을 편차로 반복한다 — 계절을 빼면 얼마나 남는가?
4. ACF로 "기억이 며칠 가는지"를 재고, 원본의 ACF에서 1년의 물결을 찾는다.
5. 평균 전략 vs naive 전략의 오차 대결 — 그리고 naive가 크게 틀리는 날은 언제인지 생각해 본다.